

Chapter 06

Synchronization Tools

Race condition - A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a race condition.

* To guard against the race condition above, we need to ensure that only one process at a time can be manipulating the variable count. To make such a guarantee, we require that the processes be synchronized in some way.

Critical Section - Let us consider a system consisting of n processes $\{P_0, P_1, P_2, \dots, P_{n-1}\}$. Each process has a segment of code, in which the process may be accessing and updating data that is shared with at least one other process, is called a critical section.

Important feature:

- When one process is executing in its critical section, no other process is allowed to execute in its critical section.

* Critical-section problem is to design a protocol that the processes can use to synchronize their activity so as to cooperatively share data.

* Each process must request permission to enter its critical section.

- The section of code implementing the request is entry section; followed by exit section

- The remaining code is remainder section

while (true) {

entry section

critical section

exit section

remainder section

Fig: General structure of process

* Critical section problem's solution must satisfy the following three requirements:

1. Mutual exclusion - If process P_i is executing in its critical section, then no other processes can be executing in their critical section.
2. Progress - If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection can not be postponed indefinitely.
3. Bounded waiting - There exists a bound, or limit, on the no. of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

* Two general approach for handling critical section problem:

1. Preemptive Kernels - allows a process to be preempted while it is running in kernel mode.

2. Non-preemptive Kernels - does not allow a process running in kernel mode to be preempted.

→ free from race condition; since one process is running

* Why favour a preemptive Kernel over a non-preemptive one?

⇒ Preemptive kernel may be more responsive; since there is less risk that a kernel mode process will run for an arbitrarily long period before relinquishing the processor to waiting process.

Furthermore, a preemptive kernel is more suitable for real time programming as it will allow a real-time process to preempt a process currently running in the kernel.

* Peterson's Solution

- is restricted to two processes that alternate execution between their critical sections and remainder sections.
- The processes are $P_0, P_1; \{P_i, P_j; j = 2 - i\}$
- requires the two processes to share two data items:

```
int turn;
boolean flag[2];
```

variable `turn` → indicates whose turn it is to enter the critical section

if `turn == i`; then P_i is allowed to enter

variable `flag` → `flag` array is used to indicate if a process is ready to enter its critical section.

if `flag[i] == true`; P_i is ready to enter

Exap - Structure of process P_i in Peterson's solution

```
while (true) {
```

```
    flag[i] = true;
```

```
    turn = i;
```

```
    while (flag[j] && turn == j)
```

```
    ;
```

```
    /* critical section */
```

```
    flag[i] = false;
```

```
    /* remainder section */
```

Explanation: To enter the critical section, process P_i first sets $flag[i]$ to be true and $turn$ to the value i .

Asserting, thereby, if P_j wants to enter the critical section, it can do so.

If both P_i and P_j wants to enter at the same time, $turn$ will be set to both i and j at the same time. Only one of these assignments will last and other will occur but will be overwritten immediately.

The eventual value of $turn$ determines which process is allowed to enter its critical section first.

* To prove that this solution is correct, we need to show that:

1. Mutual exclusion is preserved
2. The progress requirement is satisfied.
3. The bounded waiting requirement is met.

To prove property 1, each process P_i enters the critical section only when $\text{flag}[i] == \text{false}$ or $\text{turn} == i$. If both processes execute in the critical sections at the same time, then, $\text{flag}[0] == \text{flag}[1] == \text{true}$. These two observations imply that P_0 and P_1 couldn't have successfully executed their while statements at the same time, since the value of turn can be either 0 or 1, but not both. So, one of the processes will be successfully executed while the other might have to execute ~~an~~ at least one additional statement. So, mutual exclusion is preserved.

To prove properties 2 and 3, P_i can be prevented from entering the critical section only if $\text{flag}[j] == \text{true}$ and $\text{turn} == j$. If P_j is not ready to enter the critical section, $\text{flag}[j] == \text{false}$ and $\text{turn} == i$; P_i can enter the critical section then. If P_j exits the critical section, it sets $\text{flag}[j] == \text{false}$ and $\text{turn} == i$.

Thus, since P_i doesn't change the value of the variable $turn$, P_i will enter the critical section (progress) after at most one entry by P_j (bounded $\&$ waiting)

3. The bounded waiting requirement is met.

To prove property 1, each process P_i enters the critical section only when $flag[i] = true$ or $turn = i$. If both processes execute in the critical sections at the same time, then $flag[i] = flag[j] = true$. There are two observations imply that P_i and P_j couldn't have successfully executed their while statements at the same time, since the value of $turn$ can be either 0 or 1, but not both. So, one of the process will be successfully executed while other might have to execute or at least one additional statement. So mutual exclusion is preserved.

To prove properties 2 and 3, P_i can be prevented from entering the critical section only if $flag[i] = true$ and $turn = j$. If P_i is not ready to enter the critical section, $flag[i] = false$ and $turn = i$, P_i can enter the critical section then. If P_i enters the critical section, it sets $flag[i] = true$ and $turn = i$.

* Hardware Support for Synchronization

Three hardware instructions that provide support for solving the critical-section problem:

1. Memory Barriers

Memory model - How a computer architecture determines what memory guarantees it will provide to an application program is known as its memory model.

Two categories:

1. Strongly ordered - where memory modification on one processor is immediately visible to others.
2. Weakly ordered - may not be immediately visible to others.

*Memory models vary by processor types, so assumptions can't be made regarding visibility of modifications. To address this issue a memory barrier or memory fence is used.

Memory Barrier/Fence: Computer architectures provide instructions that can force any changes in memory to be propagated to all other processors, thereby ensuring that memory modifications are visible to threads running on other processors. Such instructions are known as memory barriers.

2. Hardware Instructions

- Special hardware instructions that allow us either to test and modify the content of a word or to swap the contents of two words atomically.

$\left. \begin{array}{l} \text{test_and_set()} \\ \text{compare_and_swap()} \end{array} \right\} \text{executes atomically}$

- if two `test_and_set()` or two `compare_and_swap()` instructions are executed simultaneously each on different core, they will be executed sequentially in some arbitrary order.

- Mutual exclusion implemented by declaring a boolean variable `lock`; initialized to false $\rightarrow$ `test_and_set()`
global variable `lock`; initialized to 0 $\rightarrow$ `compare_and_swap()`

3. Atomic Variables

- provides atomic operations on basic data types such as integers and booleans.
- Can be used to ensure mutual exclusion in situations where there may be a data race on a single variable while its being updated, as when a counter is incremented.
- Systems that supports atomic variables, provide special atomic data types as well as functions for accessing and manipulating atomic variables.

* Mutex Locks

- higher-level software tool to solve critical-section problem.

- Mutex $\rightarrow$ Mtual Exclusion

- A process must ~~at~~ acquire the lock before entering the critical section

acquire() function $\rightarrow$ acquires the lock

- A process releases the lock when it exits the critical section

release() $\rightarrow$ releases the lock

- A mutex lock has a boolean variable available; whose value indicates if the lock is available or not.

- If the lock is available, a call to acquire() succeeds and the lock is then considered unavailable.

- A process that attempts to acquire an unavailable lock is blocked until the lock is released.

```
while (true) {
```

```
    acquire lock
```

```
    critical section
```

```
    release lock
```

```
    remainder section
```

```
}
```

Fig: solution to critical section problem using mutex lock

Definition of acquire():

```
acquire() {
```

```
    while (!available)
```

```
        ; /* busy wait */
```

```
    available == false;
```

```
}
```

Definition of release():

```
release() {
```

```
    available = true;
```

```
}
```

* Lock Contention

Contended - A lock is considered contended if a thread blocks while trying to acquire the lock.

Uncontended - If a lock is available when a thread attempts to acquire it, the lock is considered uncontended.

High contention - a relatively large number of threads attempting to acquire the lock.
Decrease overall performance of concurrent applications.

Low contention - a relatively small no. of threads attempting to acquire the lock.

Spinlocks - the locking mechanism of choice on multiprocessor systems when the lock is to be held for a short duration.

Waiting on a lock requires 2 context switches:

1. to move the thread to the waiting state
2. to restore the waiting thread once the lock becomes available

General rule $\rightarrow$ to use a spinlock if the lock will be held for a duration of less than two context switches.

Disadvantages of Mutex locks:

- requires busy waiting
- busy waiting wastes CPU cycles

* Semaphores

A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard operations: $\text{wait}()$ and $\text{signal}()$.

Definition of $\text{wait}()$:

```
wait(s) {  
    while ( $S \leq 0$ )  
        ; // busy wait  
     $S--$ ;  
}
```

Definition of $\text{signal}()$:

```
signal(s) {  
     $S++$ ;  
}
```

- All modifications to the integer value S in the $\text{wait}()$ and $\text{signal}()$ operations must be executed atomically i.e. when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

* Semaphore Usage

Binary semaphore - The value of binary semaphore can range only between 0 and 1. Thus, binary semaphore behaves similarly to mutex lock. On systems that do not provide mutex locks, binary semaphores can be used for providing mutual exclusion.

Counting semaphore - The value of a counting semaphore can range over an unrestricted domain. This can be used to control access to a given resource consisting of a finite no. of instances.

- We can also use semaphores to solve various synchronization problems. For example, two concurrently running process P_1 and P_2 with statement S_1 and S_2 ; where S_2 is to be executed only after S_1 has completed. Letting P_1 and P_2 share a common semaphore synch; In process P_1 ,

S_1 ;

Signal(synch);

In process P_2 , $S_2 \leftarrow \text{wait(synch)}$;
 S_2 ;

* Semaphore Implementation

- To overcome busy waiting, we can modify the `wait()` and `signal()` definition.

- When a process executes `wait()` operation and finds that the semaphore value is not positive, it must wait. However, rather than engaging in busy waiting, the process can suspend itself.

- The suspended process operation places the process into a waiting queue associated with the semaphore and the ~~sem~~ state of the process is switched to the waiting state.

- A process that is suspended, waiting on a semaphore S , should be restarted when some other process executes a `signal()` operation. The process is restarted by a `wakeup()` operation, which puts the process into ready state. Then it is placed in the ready queue.

- We define semaphore as:

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Each semaphore has an integer value and a list of processes list.

- sleep() operation suspends the process and wakeup(P) resumes the execution of a suspended process P.

wait() definition:

```
wait(semaphore *s) {
```

```
    s->value--;
```

```
    if (s->value < 0) {
```

```
        add this process to s->list;
```

```
        sleep();
```

```
    }
```

```
}
```

Signal definition

```
signal(semaphore *s) {
```

```
    s->value++;
```

```
    if (s->value <= 0) {
```

```
        remove a process P from s->list;
```

```
        wakeup(P);
```

```
    }
```

```
}
```

* Monitors

When the mutex lock and semaphores are used incorrectly to solve a critical section problem, some timing errors may occur -

i) When a program interchanges the order in which the wait() and signal() operations on the semaphore mutex are executed.

signal(mutex);

critical section

wait(mutex);

ii) When a program replaces signal(mutex) with wait(mutex)

wait(mutex);

critical section

wait(mutex);

iii) When wait() or signal() or both are omitted. Then

the mutual exclusion is violated or the process will permanently block.

* Strategy for dealing with such errors is to incorporate high-level synchronization construct - the monitor type.

Monitor Usage

Abstract Data Type (ADT) - encapsulates data with a set of functions to operate on that data that are independent of any specific implementation of ADT.

Monitor type - A monitor type is an ADT that includes a set of programmer defined operations that are provided with mutual exclusion within the monitor.

- also declares the variables whose values define the state of an instance of that type, along with the bodies of the functions that operate on the variables.

Syntax of
monitor

```
monitor monitor_name {  
    /* shared variable declaration */  
    function P1(- -) {  
    }  
    :  
    function Pn(- -) {  
    }  
    initialization_code(- -) {  
    }  
}
```

Implementing a Monitor using Semaphores

- For each monitor, a binary semaphore mutex (initialized to 1) is provided to ensure mutual exclusion.
- A process must execute `wait(mutex)` before entering the monitor and must execute `signal(mutex)` after leaving the monitor.
- An additional binary semaphore `next`, initialized to 0 is used to suspend the process.
- An integer variable `next-count` is also provided to count the number of processes suspended on `next`.

`wait(mutex);`

body of F

`if (next-count > 0)`

`signal(next);`

`else`

`signal(mutex);`

Mutual exclusion within a monitor is ensured.

Resuming Processes within a Monitor

- Which of the suspended processes should be resumed next?

may use first-come, first-served (FCFS) ordering
→ not adequate for all

- conditional ~~construct~~ wait construct can be used, which has the form

$x.\text{wait}(c);$

- Example: $\text{wait}(c)$ where c is an integer expression that is evaluated when the $\text{wait}(c)$ operation is executed.

$c = \text{priority no.}$

process with smallest priority no. is resumed next.

* Liveness

- refers to a set of properties that a system must satisfy to ensure that processes make progress during their execution life cycle.

Liveness failure - A process waiting indefinitely for another process to complete its execution is called liveness failure.

- Example: Deadlock, Priority inversion

Deadlock - The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for execution of a process, when such a situation state is reached, these processes are said to be deadlocked.

Priority Inversion

- When higher priority process needs to read or modify kernel data, but a lower priority process is accessing the kernel resource, the higher priority process then has to wait as kernel is pre-empted with a task. But if the

lower priority process is preemptive in favor of another higher priority process, it becomes very complicated. This liveness problem is known as priority inversion.

- it occurs only in systems with more than two priorities.
- can be avoided by implementing a priority inheritance protocol.

Chapter 07

Synchronization Examples

* Classic Problems of Synchronization

1. Bounded Buffer Problem

2. The Readers-Writers Problem

3. The Dining-Philosophers Problem

* The Dining-Philosophers Problem

Problem Statement:

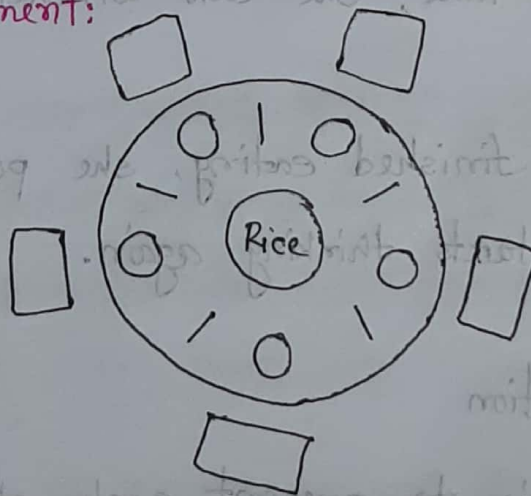


Fig: The situation of the dining philosophers

- Consider five philosophers who spend their life thinking and eating. They share a circular table surrounded by five chairs, each belonging to one philosopher.
- In center, there is a rice bowl and the table is laid with

five single chopsticks.

- When a philosopher thinks, she doesn't interact with her colleagues.
- From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her.
- A philosopher may pick one chopstick at a time. She can't pick up a chopstick that is already in the hand of a neighbor.
- When a hungry philosopher has both her chopsticks at the same time, she eats without releasing the chopsticks.
- When she is finished eating, she puts down both chopsticks and starts thinking again.

Semaphore solution

- one solution is to present each chopstick with a semaphore. `semaphore chopstick[s];`
- A philosopher tries to grab a chopstick by executing a `wait()` operation on that semaphore.
- She releases her chopsticks by executing the `signal()` operation on the appropriate semaphores.

- Although this guarantees no two neighbors ~~has~~ are eating simultaneously, it can create deadlock. When all five philosophers become hungry at the same time and each grabs her left chopstick; when they tries to grab their right chopsticks it'll be delayed forever.

- To solve the deadlock problem:

i) Allow at most four philosophers to be sitting simultaneously.

ii) Allow a philosopher to pick up her chopstick only if both chopsticks are available.

iii) Use an asymmetric solution - an odd-numbered philosopher picks up her left chopstick first and then right; whereas even numbered philosopher picks up her right chopstick first and then left.

- A deadlock free solution doesn't necessarily eliminate the possibility of starvation. One of the philosophers ~~could~~ would starve to death.

Monitors Solution

- This imposes a restriction that a philosopher may pick up her chopsticks only if both of them are available.
- To code this, we may find three states in which the philosophers might be-

Thinking, Hungry, Eating

- we also need condition `self[5]`, which allows a philosopher to delay herself when she is hungry but is unable to obtain the chopsticks she needs.
- Each philosopher must invoke the operation `pickup()` before starting to eat. After This may result in the suspension of the process. After the successful completion of the operation, the philosopher may eat.
- Following this, the philosopher invokes `putdown()` operation.

Dining-Philosophers.`pickup(i);`

eat

Dining-Philosophers.`putdown(i);`

Chapter 08

Deadlocks

Deadlock - A thread request resources; If the resources are not available at the time, the thread enters a waiting state. Sometimes, a waiting thread can never again change state, because the resources it has requested are held by other waiting threads. This situation is called a deadlock.

*System model

System - A system consists of a finite number of resources to be distributed among a number of competing threads.

Under the normal mode of operation, a thread may utilize a resource in only the following sequence:

1. **Request**: The threads request the resource. If the request can not be granted immediately, then the threads requesting thread must wait until it can acquire the resource.
2. **Use**: The thread can operate on the resource.

3. Release: The thread releases the resource after using it.

*Deadlock in Multithreaded Applications

```
pthread_mutex_t first_mutex;  
pthread_mutex_t second_mutex;
```

Here two mutex locks are created

Next two threads are created. If thread-one attempts to acquire the mutex locks in order (1) first_mutex (2) second_mutex. At the same time, thread-two attempts to acquire the mutex locks in order (1) second_mutex (2) first_mutex. Deadlock is possible if thread-one acquires first_mutex while thread-two acquires second_mutex.

Livelock

- another form of liveness failure; similar to deadlock
- occurs when a thread continuously attempts an action that fails.
- can generally be avoided by having each thread retry the failing operation at random times.

* Dead lock Characterization

Necessary conditions:

A Deadlock situation can arise if the following four conditions hold simultaneously in a system:

1. Mutual exclusion - At least one resource must be held in a nonshareable mode.
2. Hold and wait - A thread must be holding at least one resource and waiting to acquire additional resources that are currently being held by other threads.
3. No preemption - Resources can not be preempted; i.e. a resource can be released only voluntarily by the thread holding it, after that thread has completed its task.
4. Circular wait - A set $\{T_0, T_1, \dots, T_n\}$ of waiting threads must exist such that T_0 is waiting for a resource held by T_1 , T_1 for T_2 , ..., T_{n-1} for T_n and T_n is waiting for T_0 .

Resource Allocation Graph

- A graph that describes the deadlock situation.
- Consists of a set of vertices V and set of Edges E .
- Set of vertices is partitioned into two different types of nodes

$T = \{T_1, T_2, \dots, T_n\} \rightarrow$ active threads

$R = \{R_1, R_2, \dots, R_m\} \rightarrow$ resource types

Thread $T_i \rightarrow$ Resource R_j = Request edge

Resource $R_j \rightarrow$ Thread T_i = Allocation Edge

- Thread T_i is represented by a circle and resource type R_j is represented by a rectangle.

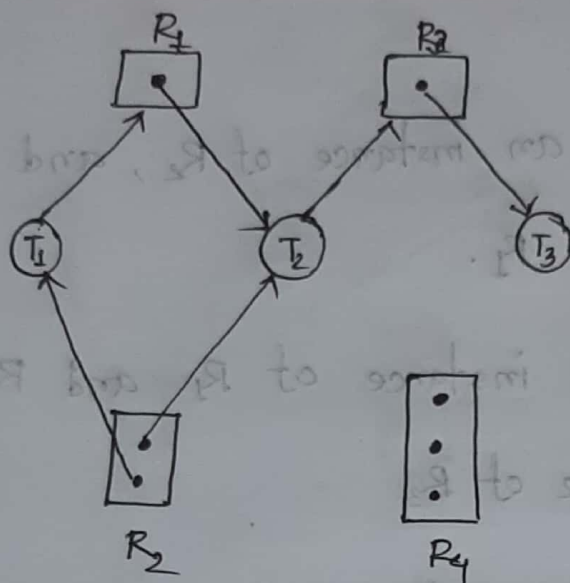


Figure - Resource Allocation Graph

This depicts the following situations:

The sets T , R , E :

$$T = \{T_1, T_2, T_3\}$$

$$R = \{R_1, R_2, R_3, R_4\}$$

$$E = \{T_1 \rightarrow R_1, R_1 \rightarrow T_2, T_2 \rightarrow R_3, R_3 \rightarrow T_3, R_2 \rightarrow T_1, R_2 \rightarrow T_2\}$$

Resource Instances:

i) 1 instance of R_1

ii) 2 " " R_2

iii) 1 " " R_3

iv) 3 " " R_4

Thread states:

i) T_1 is holding an instance of R_2 , and is waiting for an instance of R_1 .

ii) T_2 is holding an instance of R_1 and R_2 ; is waiting for an instance of R_3 .

iii) T_3 holding an instance of R_3 .

- There is no cycles in the graph, then no thread in the system is deadlocked.

- If each resource has exactly one instance type then a cycle implies that a deadlock occurred.

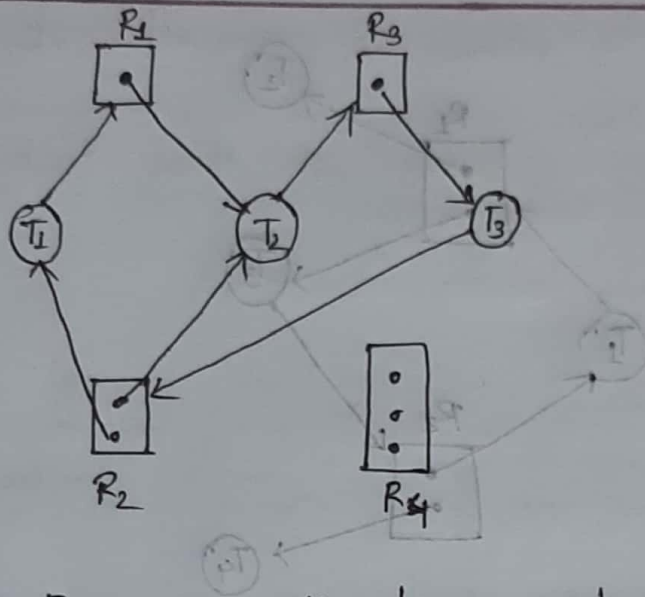


Fig: Resource allocation graph with a deadlock

- If T_3 requests an instance of R_2 , since no instance is available, we add a request edge $T_3 \rightarrow R_2$.

- At this point, two minimal cycles exists in the system:

$$T_1 \rightarrow R_1 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$$

$$T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_2$$

Here, threads T_1 , T_2 and T_3 are deadlocked.

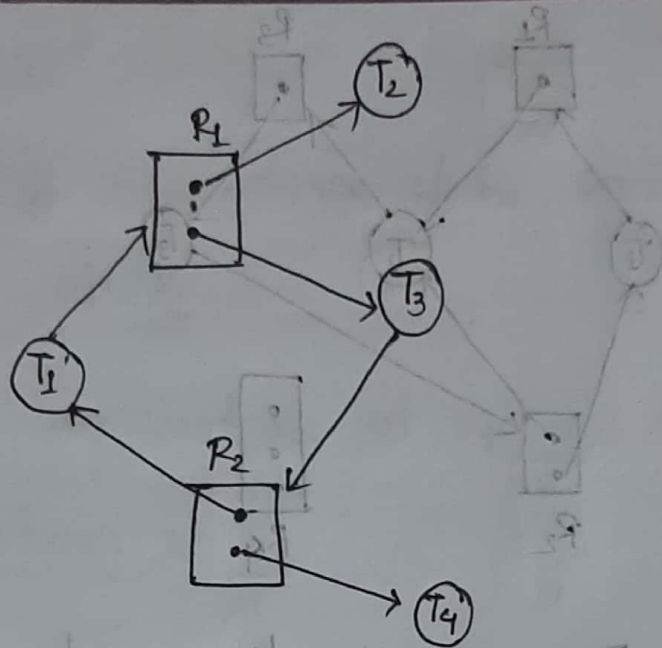


Fig: Resource allocation graph with a cycle but no deadlocks.

- Here, there's also a cycle:

$$T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$$

- However, there's no deadlock. T_4 may release the instance of R_2 and that can be allocated to T_3 , breaking the cycle.

* Methods for Handling Deadlocks

We can deal with the deadlock problem in one of three ways:

- i) We can ignore the problem altogether and pretend that deadlocks never occur in the system.
- ii) We can use a protocol to prevent or avoid deadlocks, ensuring that the system will ~~not~~ never enter a deadlocked state.
- iii) We can allow the system to enter a deadlocked state, detect it, and recover.

* Deadlock Prevention

- By ensuring that at least one of the four conditions cannot hold, we can prevent the occurrence of a deadlock.
- The mutual exclusion condition must hold. We can not prevent deadlocks by denying the mutual exclusion condition because, some resources are intrinsically nonsharable.

- To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a ~~thead~~ thread requests a resource, it does not hold any other resources.

- To ensure that the no preemption condition doesn't hold, we can use the following protocol:

If a thread is holding some resources and requests another resource that can not be immediately allocated to it, then all resources the thread is currently holding are preempted.

- The first three conditions are very impractical to prevent. The circular wait condition can be prevented by practical solutions.

One way to ensure that this condition never holds is to impose a total ordering of all resources types and to require that each thread requests resources in an increasing order of enumeration.

* Deadlock Avoidance

- A deadlock avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular wait condition can never exist.
- Resource-allocation state is defined by the number of available and allocated resources and the maximum demands of the threads.

Safe state

- A state is safe if the system can allocate resources to each thread in some order and still avoid deadlock.
- A system is in a safe state if there exists a safe sequence.
- A sequence of threads $\langle T_1, T_2, \dots, T_n \rangle$ is a safe sequence for the current allocation state if, for each T_i , the resource requests that T_i can still make, can be satisfied by the currently available resources plus the resources held by all T_j with $j < i$.

- A deadlocked state is an unsafe state.

- Not all unsafe states are deadlocks; however, an unsafe state may lead to a deadlock.

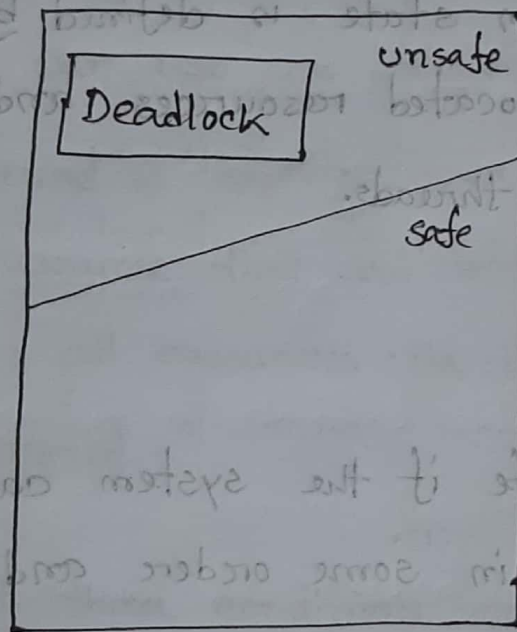


Fig: safe, unsafe and deadlocked state space

- Two algorithms

i) Resource-allocation-graph Algorithm (Single instance)

ii) Banker's Algorithm (multiple instances)

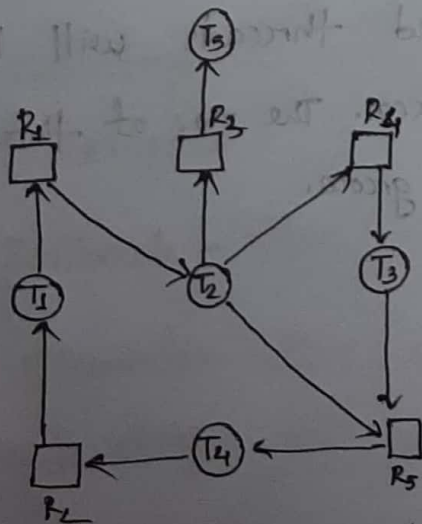
* Deadlock Detection

The system may provide:

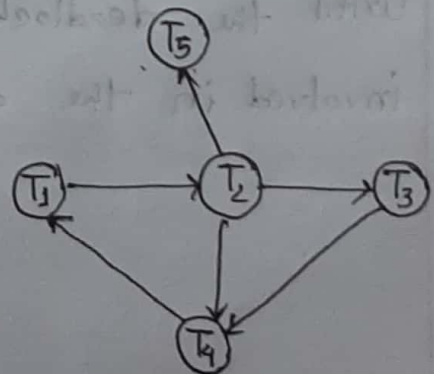
- i) An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- ii) An algorithm to recover from the deadlock.

Single Instance for each Resource Type

- uses variant of the resource-allocation graph called a wait-for graph.
- we obtain the graph by removing the resource nodes and collapsing to appropriate edges. ($T_i \rightarrow T_j$)
- Deadlock exists if there's a cycle in the graph.



Resource-allocation graph



Corresponding wait-for graph

Several Instances of a Resource Type

employs several data structures:

- i) Available - no. of available resources
- ii) Allocation - no. of resources of each type, currently allocated to each thread.
- iii) Request - current request of each thread.

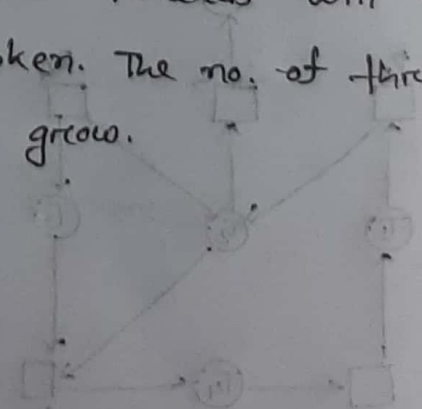
Detection Algorithm Usage

1. How often is a deadlock likely to occur?

⇒ If the deadlock occurs frequently, then the detection algorithm should be invoked frequently.

2. How many threads will be affected by deadlock when it happens?

⇒ Resources allocated to deadlocked threads will be idle until the deadlock can be broken. The no. of threads involved in the deadlock may grow.



* Recovery from Deadlock

Process and thread termination

- i) Abort all deadlocked processes - This method will clearly break the deadlock but at great expenses.
- ii) Abort one process at a time until the deadlock cycle is eliminated.

Resource Preemption

- To eliminate deadlocks, we successively preempt some resources from processes and give these to other processes until the deadlock cycle is broken.
- Three issues needs to be addressed:
 - i) Selecting a victim - which process & resource are to be preempted
 - ii) Rollback - what should be done with the preempted process
 - iii) Starvation - How can we guarantee that resources will not always be preempted from the same process